Sprawozdanie: Część 1 — Wirtualna kamera

Piotr Szepietowski (327472)

21 kwietnia 2026

1 Wymagania

Zadanie polegało na zaimplementowaniu aplikacji uruchamianej w osobnym oknie, rysującej scenę w trybie krawędziowym (wireframe) oraz implementacji wirtualnej kamery z możliwością ruchu (przód/tył, lewo/prawo, góra/dół), obrotu (yaw, pitch) oraz zmiany wartości ogniskowej („zoom").

2 Architektura

Program został podzielony na następujące warstwy:

- parsowanie danych wejściowych (krawędzie / prostopadłościany),
- reprezentacja geometrii (punkty i krawędzie),
- moduł kamery: pozycja, yaw, pitch, ogniskowa,
- rzutowanie i rysowanie (transformacja do układu kamery + rzutowanie perspektywiczne),
- pętla renderująca (wejście, aktualizacja kamery, rysowanie).

3 Implementacja kamery i rzutowania

Dla punktu w przestrzeni $P_w = (x, y, z)$ kamera posiada pozycję $C = (c_x, c_y, c_z)$ oraz orientację opisaną przez *yaw* ψ (obrót wokół osi Y) i *pitch* ϕ (obrót wokół osi X). Transformacja punktu do układu kamery przebiega następująco:

$$P_c = R_{\text{pitch}}\, R_{\text{yaw}}\, (P_w - C),$$

gdzie macierze rotacji mają postać:

$$R_{\text{yaw}} = \begin{bmatrix} \cos\psi & 0 & -\sin\psi \\ 0 & 1 & 0 \\ \sin\psi & 0 & \cos\psi \end{bmatrix}, \qquad R_{\text{pitch}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{bmatrix}.$$

Warto zaznaczyć, że powyższe macierze są *transpozycjami* (a zarazem odwrotnościami, gdyż macierze obrotu są ortogonalne: $R^{-1} = R^T$) standardowych macierzy obrotu wokół odpowiednich osi. Jest to celowy zabieg: transformujemy punkty ze przestrzeni świata *do* układu kamery, a nie odwrotnie, co wymaga zastosowania odwrotności obrotu kamery.

W implementacji, zamiast konstruować pełne macierze 3×3 i mnożyć je przez wektory, zastosowano dekompozycję obrotów i bezpośrednie obliczenia trygonometryczne jako metodę

bardziej wydajną i czytelną. Równania odpowiadające kolejnym krokom transformacji można zapisać jako:

$$\begin{aligned}
\mathrm{d}x &= x - c_x, & \mathrm{d}y &= y - c_y, & \mathrm{d}z &= z - c_z,\\
x_1 &= \mathrm{d}x\cos\psi - \mathrm{d}z\sin\psi, & y_1 &= \mathrm{d}y, & z_1 &= \mathrm{d}x\sin\psi + \mathrm{d}z\cos\psi,\\
x_2 &= x_1, & y_2 &= y_1\cos\phi - z_1\sin\phi, & z_2 &= y_1\sin\phi + z_1\cos\phi.
\end{aligned}$$

Bardziej imperatywny zapis stosowany bezpośrednio w kodzie:

```
dx = x - c_x
dy = y - c_y
dz = z - c_z
x1 = dx*cos(yaw) - dz*sin(yaw)
y1 = dy
z1 = dx*sin(yaw) + dz*cos(yaw)
x2 = x1
y2 = y1*cos(pitch) - z1*sin(pitch)
z2 = y1*sin(pitch) + z1*cos(pitch)
// P_c = (x2, y2, z2)
```

Po uzyskaniu współrzędnych w układzie kamery $P_c = (X_c, Y_c, Z_c)$ stosujemy rzutowanie perspektywiczne na ekran o rozmiarze $W \times H$:

$$x_{screen} = o_x + f\frac{X_c}{Z_c}, \qquad y_{screen} = o_y - f\frac{Y_c}{Z_c},$$

gdzie $o_x = W/2$, $o_y = H/2$ to środek ekranu, a f to parametr odpowiadający ogniskowej (w implementacji: `camera.focal`). Obecna implementacja zawiera test płaszczyzny bliskiej (near plane): jeżeli $Z_c \leq z_{near}$, punkt jest uznawany za niewidoczny, a cała krawędź go zawierająca jest pomijana. Stanowi to uproszczenie w stosunku do pełnego algorytmu klipowania (np. Cohena–Sutherlanda), który wyznaczałby punkt przecięcia krawędzi z płaszczyzną bliską; w efekcie krawędzie częściowo zasłonięte przez near plane mogą nagle znikać podczas zbliżania się kamery.

Zależność między ogniskową a *poziomym* kątem widzenia (horizontal FOV) wyraża się wzorem:

$$f = \frac{W/2}{\tan(\mathrm{FOV}/2)} \qquad \Leftrightarrow \qquad \mathrm{FOV} = 2\arctan\left(\frac{W/2}{f}\right).$$

Pionowy kąt widzenia jest pochodną poziomego i zależy od proporcji ekranu W/H.

4 Szczegóły implementacyjne

- Parsowanie plików wejściowych realizowane jest przez prosty parser tekstowy: funkcje `parseEdges` i `parseBoxes` wyodrębniają liczby typu float.
- Krawędzie przechowywane są w liście dynamicznej `std.ArrayList(Edge)`, która jest iterowana podczas renderowania.
- Do rysowania użyto biblioteki Raylib — interfejs C został zaimportowany przez `@cImport` (np. `InitWindow`, `DrawLine`).
- Preferowano bezpośrednie obliczenia trygonometryczne nad mnożeniem macierzy ze względu na wydajność w przypadku ograniczonej liczby rotacji.
- Wartość ogniskowej jest ograniczana w kodzie (zakres ok. 80–1200), co zapobiega błędom numerycznym.

- Wartość pitch jest ograniczana do zakresu $\pm 1{,}4\,\text{rad}$ ($\approx \pm 80$), co zapobiega efektowi *gimbal lock* i odwróceniu obrazu przy patrzeniu pionowo w górę lub w dół.
- Ruch kamery (W/A/S/D) odbywa się w *lokalnym* układzie kamery, tzn. wektor „do przodu"jest wyznaczany na podstawie aktualnego kąta yaw: $\mathbf{f} = (\sin\psi,\ 0,\ \cos\psi)$. Dzięki temu kamera porusza się zgodnie z kierunkiem patrzenia.

5 Translacja vs Zoom

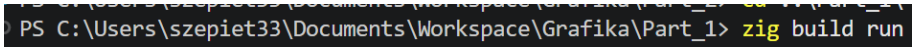
W praktyce „przesunięcie kamery"(translacja) oraz „zoom"(zmiana ogniskowej) wpływają na obraz w różny sposób:

1. **Translacja:** zmiana pozycji kamery $C \mapsto C + t$ powoduje efekt paralaksy. Przedmioty bliskie przesuwają się na ekranie szybciej niż dalekie, co może odsłaniać lub zasłaniać obiekty.
2. **Zoom (zmiana f):** nie zmienia pozycji kamery, jedynie skaluje obraz. Relacje między obiektami (paralaksa) pozostają identyczne; zmienia się jedynie wycinek obserwowanej sceny.

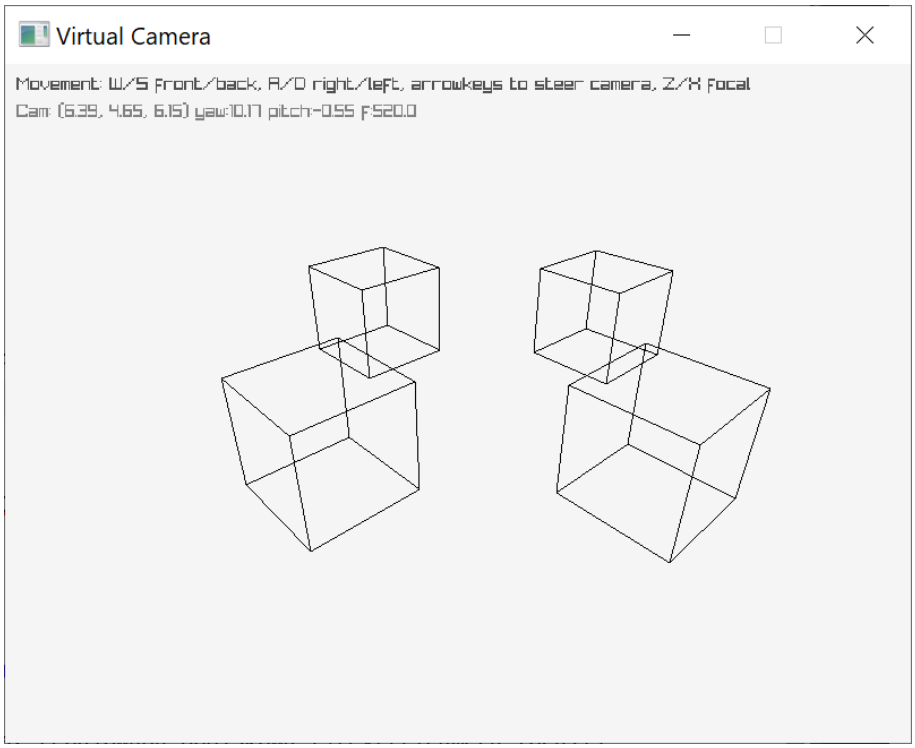
W implementacji **translacja** realizowana jest klawiszami W/A/S/D (ruch poziomy) oraz Q/E (ruch pionowy), natomiast **zoom** — klawiszami Z/X zmieniającymi wartość `camera.focal`.

Przykład numeryczny. Dla punktu $X_c = 1$: przy $Z_c = 10$ i $f = 520$ mamy $x \approx o_x + 52$, a dla $Z_c = 20$ mamy $x \approx o_x + 26$. Jeśli kamera przesunie się do przodu o 2 jednostki (translacja), nowe wartości Z_c wyniosą 8 i 18, co daje $x \approx o_x + 65$ oraz $x \approx o_x + 29$. Translacja zmienia obraz nieliniowo względem głębokości, podczas gdy zoom skaluje go równomiernie.

6 Przykłady użycia



Rysunek 1: Przykład wywołania programu bez dodatkowych argumentów.

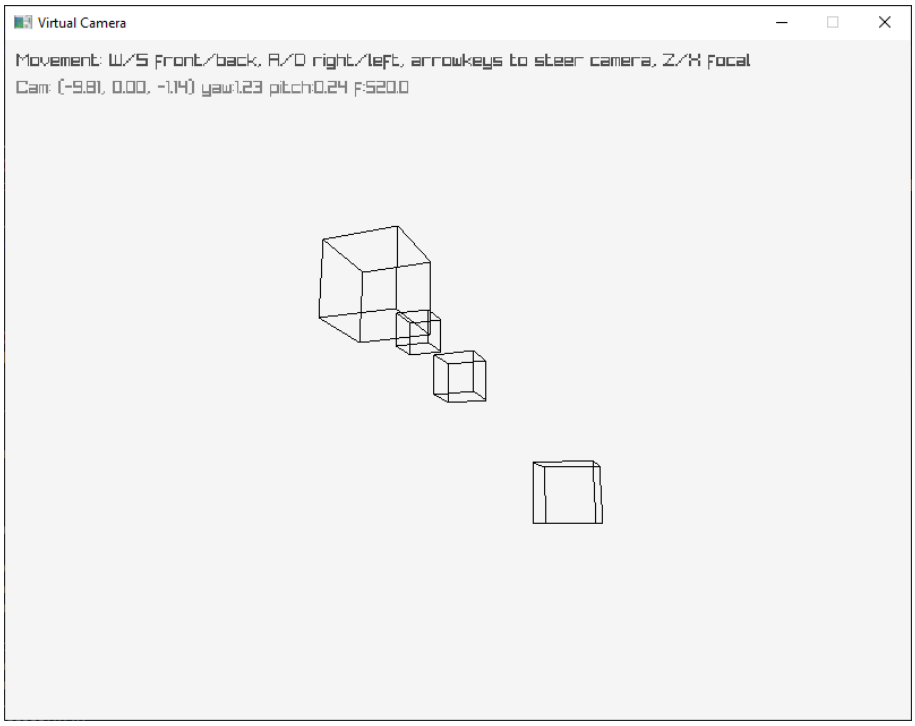


Rysunek 2: Wygląd układu domyślnego (brak pliku wejściowego).

```
PS C:\Users\szepiet33\Documents\Workspace\Grafika\Part_1> zig build run -- --boxes .\boxes.txt
```

Rysunek 3: Przykład wywołania dla jednego z załączonych plików.

Na powyższym rysunku można zaobserwować zjawisko przesłaniania dalszych figur przez te znajdujące się bliżej kamery.

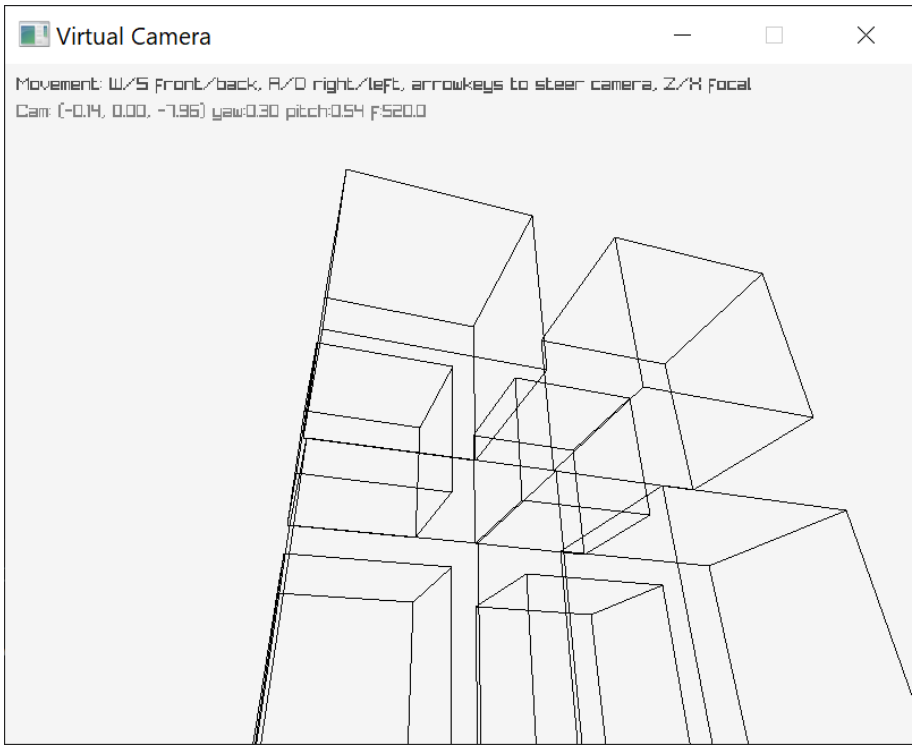


Rysunek 4: Ilustracja rzutu dla pierwszego pliku z repozytorium.

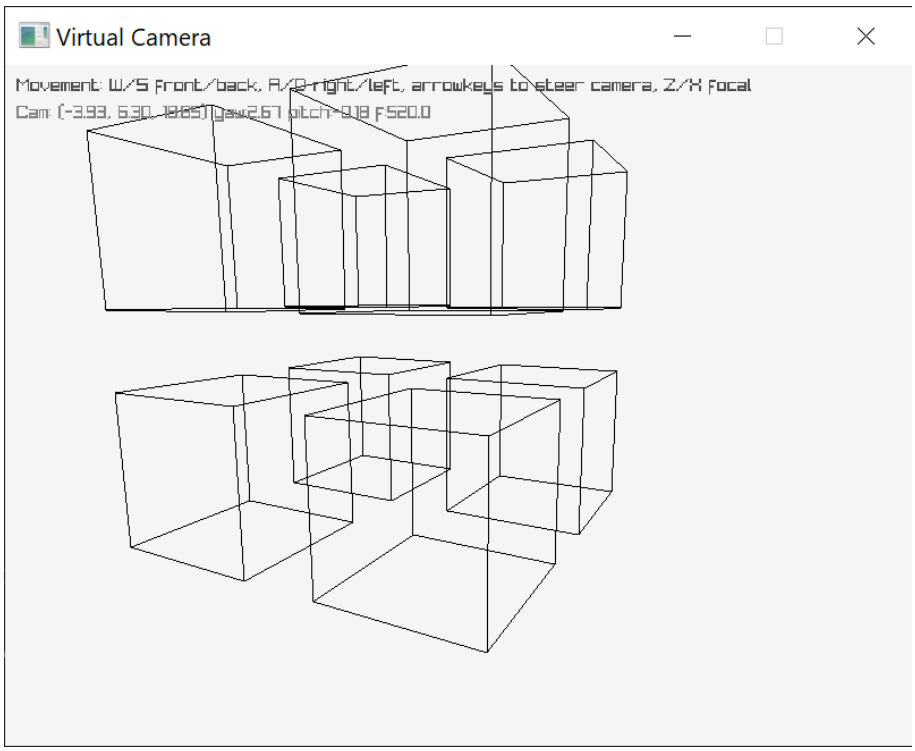
W tym przykładzie widoczne jest nakładanie się dolnych krawędzi oraz przesłanianie wyżej położonej kostki przez tę znajdującą się niżej.

Rysunek 5: Załadowany plik z układem Kostki Rubika.

Układ zainspirowany Kostką Rubika składa się z 8 kostek oddzielonych niewielką odległością, co pozwala testować poprawność rzutowania z różnych kierunków.



Rysunek 6: Widok na kostkę z pozycji startowej.



Rysunek 7: Widok na kostkę z poziomu dolnej warstwy elementów.

7 Dodatkowe rozszerzenia

7.1 Zaimplementowane

- Obsługa formatu krawędzi oraz prostopadłościanów (`addBoxEdges`).
- Pełna kontrola kamery: translacja (W/A/S/D, Q/E), yaw/pitch oraz ogniskowa.
- Test płaszczyzny bliskiej (near plane clipping).
- Ograniczenia numeryczne dla parametrów kamery (ogniskowa, pitch).

7.2 Proponowane usprawnienia

- Rotacja wokół osi Z (roll).
- Interfejs oparty na kącie FOV zamiast bezpośredniej wartości f.
- Pełne klipowanie krawędzi na płaszczyźnie bliskiej (algorytm Cohena–Sutherlanda).
- Wczytywanie formatów siatek 3D (np. OBJ).
- Optymalizacje: backface culling, Z-buffer, przeniesienie obliczeń na GPU (shadery).

8 Wnioski

Projekt pozwolił na praktyczne zapoznanie się z transformacjami 3D oraz integracją języka Zig z biblioteką Raylib. Implementacja oparta na bezpośrednich obliczeniach trygonometrycznych okazała się wydajna i wystarczająca dla wizualizacji typu wireframe.

9 Repozytorium

`https://github.com/Hadidomena/Grafika`